

Question Bank — Week 9

Basic sorting (Lecture 09) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice	1
Part B — Short answer and essay	3
Part C — Trace the algorithms	4
Part D — Array state	6
Part E — Complexity analysis	7
Part F — Find and fix the bug	8
Part G — Write the code	8

Questions: [week09-questions.md](#). Commit to your own answer before reading one here. Every trace and count below was produced by running the reference code.

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	c	M07	b	M13	a	M19	a
M02	a	M08	d	M14	b	M20	a
M03	d	M09	c	M15	c	M21	b
M04	a	M10	b	M16	c	M22	c
M05	c	M11	d	M17	b		
M06	d	M12	b	M18	d		

W9-M01 — c. Stability is about equal keys and their arrival order. (a) is “in place”; (d) is a property of selection sort.

W9-M02 — a. The course sorts return a new list, but they rearrange their working Array with $O(1)$ extra slots — so they are in place in this sense. (b) is what Python’s `list.sort` does; it is not the definition.

W9-M03 — d. Its long-distance swap can jump an element over an equal one. Bubble and insertion sort move a value only past **strictly** greater ones; a merge that takes from the left on a tie is stable (Week 10).

W9-M04 — a. The first pass compares the 5 neighbour pairs, swaps nothing, and the early exit stops the sort: $n - 1 = 5$.

W9-M05 — c. $5 + 4 + 3 + 2 + 1 = 15 = n(n - 1)/2$. Selection sort scans the whole rest in every round, sorted or not.

W9-M06 — d. The 9 is carried to the end; the pass swaps 5 and 2, then 9 and 1, 9 and 7, 9 and 3. (a) is the first round of **selection** sort; (c) is after the second pass.

W9-M07 — b. The minimum, 1 at index 3, is swapped with the 5 at index 0. (a) is bubble sort's first pass.

W9-M08 — d. Before that round the prefix is [2, 5, 9]; the 1 shifts all three right and lands at index 0.

W9-M09 — c. (3, 1) and (3, 2). The pair (1, 2) is in order.

W9-M10 — b. Each shift moves current past one larger value before it — one inverted pair — and changes the order of no other pair (W9-E3).

W9-M11 — d. One comparison per round and no shifts: $n - 1$ comparisons.

W9-M12 — b. At most $n - 1$ swaps, whatever the input. It is not stable (c), gains nothing from sorted input (a), and (d) is counting sort.

W9-M13 — a. At most one swap per round, and there are $n - 1$ rounds. 45 (b) is the number of comparisons.

W9-M14 — b. $k = \max + 1 = 5$: one counter for each value 0, 1, 2, 3, 4.

W9-M15 — c. One pass to count (n), one over the counters (k), n writes.

W9-M16 — c. Sorting two values up to a million needs a million counters and a loop over all of them.

W9-M17 — b. A decision tree with at least $n!$ leaves has height at least $\log_2 n!$, which is $\Theta(n \log n)$.

W9-M18 — d. $4! = 24$ orders, and $2^4 = 16 < 24 \leq 32 = 2^5$: at least $\lceil \log_2 24 \rceil = 5$ comparisons in the worst case.

W9-M19 — a. Two different input orders must end at different leaves, and yes/no answers can tell apart at most 2^h cases.

W9-M20 — a. Every frame holds a reference to the **same** list; by the time you look, it is sorted. `step_slider` hides the bug by copying each frame as it arrives; `list(generator)` does not.

W9-M21 — b. $n^2/2$ copies of n values each: $\Theta(n^3)$. The reference fixes it with `snapshot=_live` in the plain sort (Lecture 09, "The price of watching").

W9-M22 — c. Reversed input has every pair inverted: $6 \times 5/2 = 15$ inversions, and bubble sort makes one swap per inversion.

Part B — Short answer and essay

W9-E1 (4)

- **Stable:** elements with equal keys come out in the order they went in. **In place:** only $O(1)$ memory besides the array being sorted.
- **Bubble sort** swaps neighbours only when $a[j] > a[j + 1]$ — strictly — so two equal values are never swapped and never pass each other: **stable**. **Insertion sort** shifts only values $>$ current, so current stops at an equal key: **stable**. **Selection sort** swaps $a[i]$ with a minimum far to its right, which can jump $a[i]$ over an equal value: **not stable**. All three are in place.
- **Example:** cards 2a, 2b, 1c compared by number. Round 0 swaps 1c with 2a: 1c, 2b, 2a — 2a and 2b have changed order.
- **Two keys:** to sort by grade, and by name within a grade, sort by name first, then **stably** by grade. The second sort leaves students with equal grades in name order. An unstable second sort scrambles them.

W9-E2 (4)

	comparisons, best	comparisons, worst	swaps or shifts
bubble (early exit)	$n - 1$ (sorted)	$n(n - 1)/2$	I swaps (inversions)
selection	$n(n - 1)/2$	$n(n - 1)/2$	at most $n - 1$ swaps
insertion	$n - 1$ (sorted)	$n(n - 1)/2$	I shifts

- **Insertion sort** for small or nearly sorted data, or when stability is needed: it adapts to the input ($O(n + I)$) and does about half the comparisons of bubble sort on random data.
- **Selection sort** when writes are expensive (large records, flash memory): it writes $O(n)$ times whatever the input.
- **Bubble sort** has no real niche — insertion sort is at least as good in every case. Its one merit is teaching: with the early exit it detects sorted input in one pass, and “neighbours swap” is the easiest sort to explain and to animate.

W9-E3 (4)

- **Inversion:** a pair of positions $i < j$ with $a[i] > a[j]$.
- **One shift per inversion:** when current is inserted, it moves left past exactly the values before it that are larger — each such pair was an inversion, and after the move it is not. The relative order of every other pair is unchanged (the shifted values keep their order among themselves). So each shift removes exactly one inversion, and the sort ends when none are left: shifts = I , and comparisons $\leq I + n - 1$.

- **Cases:** sorted, $I = 0$: $\Theta(n)$. Reversed, $I = n(n-1)/2$: $\Theta(n^2)$. Random: each pair inverted with probability $\frac{1}{2}$, so the expected I is $n(n-1)/4 = \Theta(n^2)$, half the worst.
- **Libraries:** real data is often nearly sorted, and short pieces are common inside divide-and-conquer sorts. On both, insertion sort is near-linear, has a tiny constant, needs no memory and is stable. Timsort (Python, Java) extends short runs with binary insertion sort; introsort (C++) finishes small pieces with insertion sort.

W9-E4 (4)

- **What it yields:** (values, highlight) — a snapshot of the array and the indices the next step concerns. Bubble sort yields once at the start, before every comparison, and once at the end.
- **Reuse:** the plain sort runs the generator to the end and keeps the last state (`_finish`). One implementation: the animation shows exactly the code the tests grade.
- **Rules:** each frame must be its **own** list(`list(a)`), or the frames all show the final state (`test_step_frames_are_independent_snapshots`); and the last frame must be sorted (`test_step_form_is_a_generator_ending_sorted`).
- **The price of watching:** a copy costs $O(n)$; made before each of $n^2/2$ comparisons, it makes the plain sort $\Theta(n^3)$ — measured, the time grows about $8\times$ per doubling instead of $4\times$. **Fix:** a snapshot parameter, `list` by default for animations; the plain sort passes `_live`, which returns the working Array without copying, and `_finish` makes one copy at the end.

W9-E5 (4)

- **Bound:** every sort that learns about its input **only by comparing** two elements needs at least $\lceil \log_2 n! \rceil$ comparisons in the worst case, and $\log_2 n! = \Theta(n \log n)$.
- **Argument:** draw the sort's comparisons as a binary decision tree; one run is a root-to-leaf path. Different input orders need different rearrangements, so they must reach different leaves: at least $n!$ leaves. A binary tree of height h has at most 2^h leaves, so $h \geq \log_2 n!$. Since $n! \geq (n/2)^{n/2}$, $\log_2 n! \geq \frac{n}{2} \log_2 \frac{n}{2}$. For $n = 10$: at least 22 comparisons.
- **Counting sort:** does not compare. `counts[v] += 1` uses the value as an address — one step with k outcomes, not 2 — so the bound, which is about comparison sorts only, does not apply. It pays with $O(k)$ memory and with keys restricted to small non-negative integers.
- **When:** integer keys whose range k is at most about n — marks, ages, letters, grades. Not for phone numbers, floats or strings.

Part C — Trace the algorithms

W9-T1

start	3	8	1	6	2	
after pass 1	3	1	6	2	8	4 comparisons, 3 swaps
after pass 2	1	3	2	6	8	3 comparisons, 2 swaps

```
after pass 3    1  2  3  6  8    2 comparisons, 1 swap
after pass 4    1  2  3  6  8    1 comparison,  0 swaps: stop
```

10 comparisons and 6 swaps; the input has 6 inversions. Pass 4 is the last possible pass anyway, so the early exit saves nothing here.

W9-T2

Round	Minimum of the rest	Swap?	Array after
i = 0	10 at index 1	29 with 10	[10, 29, 14, 37, 13]
i = 1	13 at index 4	29 with 13	[10, 13, 14, 37, 29]
i = 2	14 at index 2	no	[10, 13, 14, 37, 29]
i = 3	29 at index 4	37 with 29	[10, 13, 14, 29, 37]

$4 + 3 + 2 + 1 = 10$ comparisons, 3 swaps.

W9-T3

```
start          8  4  6  2  9  1
i = 1          4  8  6  2  9  1    1 shift,  1 comparison
i = 2          4  6  8  2  9  1    1 shift,  2 comparisons
i = 3          2  4  6  8  9  1    3 shifts, 3 comparisons
i = 4          2  4  6  8  9  1    0 shifts, 1 comparison
i = 5          1  2  4  6  8  9    5 shifts, 5 comparisons
```

10 shifts and 12 comparisons. The inversions: 8 is before 4, 6, 2 and 1 (4); 4 before 2 and 1 (2); 6 before 2 and 1 (2); 2 before 1 (1); 9 before 1 (1) — **10**, the number of shifts. Rounds 1, 3 and 5 walk off the front, so they make as many comparisons as shifts; the others make one more, the comparison that stops the walk.

W9-T4

- **Bubble: 12.** One yield at the start, one before each comparison, one at the end. Reversed input swaps in every pass, so all four passes run: $4 + 3 + 2 + 1 = 10$ comparisons. $1 + 10 + 1 = 12$.
- **Selection: 14.** One at the start, one before each of the $4 + 3 + 2 + 1 = 10$ comparisons, one after each swap, one at the end. Round 0 swaps 5 with 1 ([1, 4, 3, 2, 5]), round 1 swaps 4 with 2 ([1, 2, 3, 4, 5]), rounds 2 and 3 swap nothing: 2 swaps. $1 + 10 + 2 + 1 = 14$.
- **Insertion: 12.** One at the start, one after each **shift**, one at the end. The shifts are the inversions: 10 in a reversed list of 5. $1 + 10 + 1 = 12$.

(Checked with `len(list(sorting.bubble_sort_steps([5, 4, 3, 2, 1])))` and its two siblings, on the reference solution.)

W9-T5

- $k = 5 + 1 = 6$. `counts = [2, 0, 2, 3, 0, 1]` (indices 0 to 5): two 0s, no 1, two 2s, three 3s, no 4, one 5.

- Output: [0, 0, 2, 2, 3, 3, 3, 5].
- **Starting positions** (a running sum of the counts before each value): 0 starts at 0, 1 at 2, 2 at 2, 3 at 4, 4 at 7, 5 at 7.
- The three 3s are placed in input order at positions 4, 5 and 6: the **first** 3 of the input (index 2) goes to **4**, the last (index 7) to **6**. Equal keys keep their order — that is the stability.

Part D — Array state

W9-S1 (shaded values in brackets)

start	6	5	3	1	8	7	2	4	
after pass 1	5	3	1	6	7	2	4	[8]	7 comparisons, 6 swaps
after pass 2	3	1	5	6	2	4	[7]	8]	6 comparisons, 4 swaps
after pass 3	1	3	5	2	4	[6]	7	8]	5 comparisons, 3 swaps
after pass 4	1	3	2	4	[5]	6	7	8]	4 comparisons, 2 swaps
after pass 5	1	2	3	[4]	5	6	7	8]	3 comparisons, 1 swap
after pass 6	[1]	2	3	4	5	6	7	8]	2 comparisons, 0 swaps: stop

6 passes, 27 comparisons, 16 swaps — the input has 16 inversions. The early exit saves the seventh pass (1 comparison). After pass 5 the array is already sorted, but the sort cannot know that until a pass makes no swap.

W9-S2

frame 1	[5, 1, 4, 2, 8]	()	the start
frame 2	[1, 5, 4, 2, 8]	(0, 1)	i = 1: 1 shifts past 5
frame 3	[1, 4, 5, 2, 8]	(1, 2)	i = 2: 4 shifts past 5
frame 4	[1, 4, 2, 5, 8]	(2, 3)	i = 3: 2 shifts past 5
frame 5	[1, 2, 4, 5, 8]	(1, 2)	i = 3: 2 shifts past 4
frame 6	[1, 2, 4, 5, 8]	()	the end (i = 4: 8 does not move)

Six frames: the start, one per shift (4 shifts = 4 inversions), the end. Each frame is a permutation because the reference writes current into the hole after **every** shift ($a[j + 1] = \text{current}$ inside the loop). A textbook insertion sort writes current once, after the walk, and a frame taken in between would show the shifted value twice — [5, 5, 4, 2, 8] — and the 1 missing.

W9-S3

Step	values[mid]	Action	Array after	lo	mid	hi
start			[2, 0, 2, 1, 1, 0]	0	0	5
1	2	swap with hi; hi - 1	[0, 0, 2, 1, 1, 2]	0	0	4
2	0	swap with lo; both + 1	[0, 0, 2, 1, 1, 2]	1	1	4
3	0	swap with lo; both + 1	[0, 0, 2, 1, 1, 2]	2	2	4
4	2	swap with hi; hi - 1	[0, 0, 1, 1, 2, 2]	2	2	3

Step	values[mid]	Action	Array after	lo	mid	hi
5	1	mid + 1	[0, 0, 1, 1, 2, 2]	2	3	3
6	1	mid + 1	[0, 0, 1, 1, 2, 2]	2	4	3

mid > hi: the unseen region is empty — stop. Six steps for six values; each step shrinks the unseen region values[mid..hi] by one. In step 1, the value that arrives at mid from the back (a 0) has not been looked at, which is why mid does not advance.

Part E — Complexity analysis

W9-K1

- **Worst case** $\Theta(n^2)$: insertion sort on reversed (or random) input; the scan of neighbours is only $\Theta(n)$ after it.
- **Best case** $\Theta(n)$: sorted input with no duplicates — insertion sort makes $n - 1$ comparisons, and the scan finds nothing in $n - 1$. (With a duplicate early in a sorted list, the scan can stop sooner, but the sort already cost $\Theta(n)$.)
- **With counting_sort**, values in $0 \dots k - 1$: $\Theta(n + k)$ — linear when k is $O(n)$. (A duplicate is then simply a count of 2 or more; the sort is not even needed.)
- **Two nested loops** over all pairs: $\Theta(n^2)$ in the worst case (no duplicates) — no better than the sort in the worst case, and it has no $\Theta(n)$ best case on duplicate-free input.

W9-K2

- Round i compares $a[\text{smallest}]$ with each of $a[i+1 \dots n-1]$: $n - 1 - i$ comparisons. Total $\sum_{i=0}^{n-2} (n - 1 - i) = (n - 1) + \dots + 1 = n(n - 1)/2$, **for every input**. Swaps: at most one per round — at most $n - 1$.
- **Cost with a write = 100 comparisons**: selection $499,500 + 1,986 \times 100 = 698,100$; insertion $249,614 + 498,241 \times 100 = 50,073,714$. Selection sort is about **72 times cheaper**, although it makes twice the comparisons. Which sort is “faster” depends on what an operation costs.

W9-K3

- Let x be at index i , with sorted position p , $|i - p| \leq k$. Let L be the number of larger values before x , and S the number of smaller values after it. Then $p = i - L + S$.
- If $S = 0$: $L = i - p \leq k$.
- If $S > 0$: every smaller value after x has sorted position below p , so it sits at an index at most $p - 1 + k$ — all S of them in the indices $i + 1 \dots p + k - 1$, so $S \leq p + k - 1 - i$. Then $L = i - p + S \leq i - p + (p + k - 1 - i) = k - 1$.
- So every value has **at most k** larger values before it, shifts at most k times, and the sort makes $I \leq nk$ shifts and at most $n(k + 1)$ comparisons: $O(nk)$.
- **$k = 1$** : $O(n)$ — only neighbours are out of order. **$k = n$** : $O(n^2)$ — no information, the general worst case.

Part F — Find and fix the bug

W9-B1. `range(end + 1)` lets `j` reach `end`, and then `a[j + 1]` is `a[end + 1]` — past the end on the first pass, where `end = n - 1`: `IndexError: Array index 5 out of range for length 5 on [5, 2, 9, 1, 7]`. It fails 10 of the 12 bubble tests; only `[]` and `[1]` pass, because their loops never run. **Fix:** `for j in range(end):` — the last pair of the pass is `(end - 1, end)`.

W9-B2. It swaps **every** time it meets a value smaller than the current `a[i]`, instead of remembering the index of the minimum and swapping once after the scan. It still sorts — after round `i`, `a[i]` holds the minimum of the rest — so the tests pass, but it throws away selection sort's only virtue. On `[6, 5, 4, 3, 2, 1]` it makes **15 swaps**; the real selection sort makes **3** (6 with 1, 5 with 2, 4 with 3). **Fix:** keep `smallest`, compare with `a[smallest]`, and swap `a[i]` with `a[smallest]` once, after the inner loop, only if they differ.

W9-B3. `a[j] >= current` walks current past values **equal** to it, so equal keys swap order: **not stable**. On plain numbers nobody can see it, so every test passes except `test_insertion_sort_is_stable: assert ['0d', '0b', '1c', '1a'] == ['0b', '0d', '1a', '1c']`. It also does extra shifts on every run of equal values. **Fix:** `a[j] > current`.

W9-B4. Two bugs.

- `k = max(values)` makes the counters `0 .. max - 1`, so `counts[max]` does not exist: `counting_sort([3, 1, 4, 1, 5, 0])` raises `IndexError: Array index 5 out of range for length 5`. (And the write-out loop, `range(k)`, would never write the largest value.) **Fix:** `k = max(values) + 1`.
- The empty list: `max([])` raises `ValueError: max() iterable argument is empty`. **Fix:** `return []` first.

The reference also checks that every value is a non-negative int, and accepts `max_value` to skip the max pass.

Part G — Write the code

W9-C1

```
def count_inversions(values):
    n = len(values)
    count = 0
    for i in range(n):
        for j in range(i + 1, n):
            if values[i] > values[j]:
```



```

        count += 1
    return count

```

Every pair once: $n(n-1)/2$ comparisons, $\Theta(n^2)$. **Why it equals bubble sort's swaps:** a swap of two neighbours changes the order of exactly one pair — the two neighbours — and bubble sort swaps only when that pair is inverted. So each swap removes one inversion, and the sort ends with none: swaps = I. Week 10 counts inversions in $O(n \log n)$, inside merge sort.

W9-C2

```

def sort_by_key(items, key):
    out = list(items)
    for i in range(1, len(out)):
        current = out[i]
        k = key(current)
        j = i - 1
        while j >= 0 and key(out[j]) > k: # strictly greater: stable
            out[j + 1] = out[j]
            j -= 1
        out[j + 1] = current
    return out

```

Why keys only: the items may not be comparable at all — the test sorts Record objects with no $<$ — and even when they are, comparing whole items would sort by something other than the key: two tuples with the same key would be ordered by their second field, and the stability test could not tell. $>$ (not $\geq$) is what keeps it stable. It calls key once per comparison; a faster version computes each key once and keeps it beside its item.

W9-C3

```

def dutch_flag(values):
    lo, mid, hi = 0, 0, len(values) - 1
    # values[:lo] are 0, values[lo:mid] are 1, values[hi+1:] are 2
    while mid <= hi:
        v = values[mid]
        if v == 0:
            values[lo], values[mid] = values[mid], values[lo]
            lo += 1
            mid += 1
        elif v == 1:
            mid += 1
        else:
            values[mid], values[hi] = values[hi], values[mid]
            hi -= 1 # do not advance mid: new value unseen

```

Why mid stays: the value swapped in from hi comes from the unseen region; it could be a 0, a 1 or a 2, and must be looked at next. The value swapped in from lo, by contrast, is a 1 (or

`lo == mid`), already seen — so both can advance. Each step shrinks `values[mid..hi]` by one: at most n steps, $O(n)$, and $O(1)$ extra space. (Counting the 0s, 1s and 2s and writing them back is also $O(n)$ and passes the tests, but takes two passes — and would not work if the 0s, 1s and 2s were keys of records with other data attached; the Dutch flag moves the records themselves.)

W9-C4

```
def sort_k_sorted(values, k):
    out = list(values)
    for i in range(1, len(out)):
        current = out[i]
        j = i - 1
        while j >= 0 and out[j] > current:
            out[j + 1] = out[j]
            j -= 1
        out[j + 1] = current
    return out
```

It is plain insertion sort: k is not used by the code, only by the analysis. Each value has at most k larger values before it (W9-K3), so each round makes at most k shifts and $k + 1$ comparisons: at most $n(k + 1)$ comparisons — exactly the bound the test checks, on 2,000 values with $k = 3$ (the reference makes 3,456; an $O(n^2)$ sort would make about a million).

W9-C5

```
def counting_sort_by_key(items, key, k):
    counts = Array(k, fill=0)
    for item in items:
        kk = key(item)
        if not 0 <= kk < k:
            raise ValueError(f"key {kk!r} is outside 0..{k - 1}")
        counts[kk] += 1
    start = 0 # counts[v] becomes where v starts
    for v in range(k):
        counts[v], start = start, start + counts[v]
    out = Array(len(items))
    for item in items: # input order: this is the stability
        kk = key(item)
        out[counts[kk]] = item
        counts[kk] += 1
    return list(out)
```

Three passes: count the keys; turn each count into the **starting position** of its key (a running sum — for banana, a starts at 0, b at 3, n at 4); then place each item at the next free position of its key. $O(n + k)$ time, $O(n + k)$ extra space, and no item is ever compared with another. **Why input order:** items with the same key are placed at consecutive positions in the order the loop meets them. Going through the input front to back puts the first one

first — stable. Going back to front would reverse every group of equal keys. This stable counting sort is the step that radix sort repeats once per digit.